Technical Approach Document

CardioTrack CT-200 Document Hierarchy, Versioning & QA Test Traceability Pipeline

Assignment: AI Engineering Internship Assignment

Company: Tri9T AI

Target Device: CardioTrack CT-200 Blood Pressure Monitor

Tech Stack: FastAPI, Pydantic, SQLAlchemy, SQLite, NoSQL Document Store

---

1. Executive Summary & Problem Framing

In regulated medical device engineering (governed by **FDA 21 CFR Part 820**, **IEC 62304**, and **ISO 14971**), bidirectional traceability between software requirements and verification test cases is mandatory. A prevalent, critical failure mode in medical QA is **stale traceability**: when specifications evolve (e.g. cuff emergency relief time reduced from 2.0s to 1.5s), but QA test cases generated against earlier revisions are never flagged as outdated, creating false confidence in patient safety.

This project delivers a complete backend system and REST API that:

- Parses real-world, irregular engineering PDFs into an exact hierarchical tree without data loss or incorrect parent-child linkages.
- Supports non-destructive multi-version ingestion (v1 and v2), mapping canonical logical nodes and computing delta diffs.
- Maintains version-pinned selections that freeze text snapshots and content hashes.
- Generates concrete, executable medical QA test cases using LLMs with Pydantic structured output validation, JSON repair, and deterministic fallback.
- Executes live staleness and semantic impact analysis at retrieval time, differentiating critical numerical threshold updates from cosmetic edits.

2. OCR & Document Parsing Architecture

2.1 Selected Parsing Approach: Typography- & Layout-Aware Span Extraction

Rather than treating the PDF as a flat string (discarding structural cues) or employing heavyweight, non-deterministic vision models (which hallucinate in clean vector PDFs), we engineered a **low-level text span extraction pipeline using pure-Python `pypdf` visitor callbacks**.

> **Key Architectural Advantage:** Vector engineering manuals embed exact typographic metadata: font family, font size, transformation matrices (coordinates `x, y`), and bounding boxes. Extracting these directly yields 100% deterministic precision, instantaneous execution (< 1s for the entire document), zero OCR character confusion, and zero external binary DLL dependencies.

2.2 Two-Pass Hierarchy Reconstruction Strategy

1. **Spatial Span Clustering:** Text spans on each page are sorted vertically by `y` ascending and horizontally by `x` ascending. Spans sharing the same vertical baseline (`|y1 − y2| ≤ 3.5 pt`) are

grouped into single line records.

2. **Typographic Classifier:**
   - `Title:` `font_size ≥ 20.0 pt` (Nimbus Sans Bold).
   - `Heading 1 (Sections 1–8):` `font_size ≥ 16.0 pt`, `font_weight = Bold`, regex `^\d+\.\s+`.
   - `Heading 2 (Subsections):` `font_size = 12.9 pt`, `font_weight = Bold`, regex `^\d+\.\d+\s+`.
   - `Anomalous Headings (e.g. 3.2, 2.1.1.1):` Detected by `Bold` weight matching the recursive numerical pattern `^\d+(\.\d+)+\s+`.
   - `Body Numbered Lists:` Regular font weight (`Nimbus Sans Regular`), preventing list items from being misclassified as section headers.
3. **Prefix-Based Ancestor Chain Resolution:** For section $S = p_1.p_2...p_k$, the builder searches for parent $p_1.p_2...p_{k-1}$. If missing (e.g. `2.1.1.1` skipping `2.1.1`), it walks up the prefix tree to find `2.1`, parenting `2.1.1.1` correctly under `2.1 General Specifications`.

3. Handling Document Irregularities & Structural Inconsistencies

Manual inspection and automated typography auditing of `ct200_manual.pdf` and `ct200_manual_v2.pdf` identified 6 structural edge cases:

Irregularity	Occurrence in PDF	Naive Parser Failure	Pipeline Solution
Duplicate Section Titles	`4.2 Error Codes` vs `7.1 Error Codes`	Overwrites node records or conflates parent sections.	Distinct canonical keys (`sec_4_2` vs `sec_7_1`) and parent resolution (4 vs 7).
Out-of-Order Numbering	`3.4 Auto Shutoff` appears before `3.3 Result Display`	Sequential sorting distorts document flow or mis-parents text.	Nodes are appended in physical document occurrence order (`[3.1, 3.2, 3.4, 3.3]`).
Skipped Hierarchy Levels	`2.1 General Specs` → `2.1.1.1 Battery Life` (skips `2.1.1`)	Strict level ($L-1$) checks create empty dummy nodes or drop the subsection.	Prefix ancestor truncation: walks up tree from `2.1.1` to `2.1` , linking cleanly.
Body Numbered Lists	Section 3.3 contains: `1. Normal ...` through `5. Crisis ...`	Regex matching `^\d+\.` parses list items as top-level sections 1–5.	Typography weight check: Section headings are Bold; list items are Regular and kept in body.
Multi-Line Table Cells	Table 4.2: Codes `E2` and `E5` have descriptions spanning 2–3 lines.	Table cells split across rows; columns become desynchronized.	Spatial horizontal binning ($x < 50$, $50 \le x < 180$, $x \ge 180$) merges multi-line runs into Markdown tables.
Pagination Shift Whitespace	v2 addition of Section `5.3 Data Export` pushed subsequent sections across pages.	Inter-page newline gaps produce false-positive hash mismatches on unchanged text.	Whitespace and paragraph continuation normalization preserves identical hashes for unchanged sections.

4. Multi-Version Document Matching & Diffing Strategy

When re-ingesting a revised manual (such as `data/ct200_manual_v2.pdf`), version 1 is preserved intact. Nodes are mapped across versions via **canonical logical IDs** (`logical_id` , e.g. `sec_3_2` , `sec_4_2`).

- **UNCHANGED:** Node exists in both versions with identical SHA-256 content hashes ($H_{v1} == H_{v2}$).
- **MODIFIED:** Node exists in both versions with different hashes ($H_{v1} e H_{v2}$). Line-level unified diffs are generated.
- **ADDED:** Logical ID present in v2 but not v1 (e.g. `5.3 Data Export`).
- **DELETED:** Logical ID present in v1 but omitted in v2.

> **Known Failure Modes of this Matching Strategy:**

> 1. **Complete Section Renumbering:** If an author inserts a new section 2.1 and shifts old 2.1 to 2.2, a pure section-number matcher matches old 2.1 with new 2.1, reporting a false 100% diff.
> 2. **Section Splitting / Merging:** If one section is split into two, a 1:1 logical ID cannot capture 1-to-many relationships without semantic embeddings.

5. Version-Pinned Selections

When a QA engineer creates a named selection (e.g. `POST /api/selections`), the system freezes an immutable snapshot of:

- The exact requirement heading and body text.
- The SHA-256 content hash at the time of selection.
- The pinned document version (e.g. `v1`).

If version 2 is subsequently ingested, existing selections continue resolving to their original frozen text snapshots.

6. LLM QA Test Generation & Error Handling Strategy

6.1 Prompt Engineering for Medical Device QA

The system prompt enforces compliance with **ISO 14971 (Risk Management)** and **IEC 62304 (Medical Device Software Life Cycle)**:

- Every test case must trace directly to quantitative thresholds, error codes, and timing constraints.
- Steps must be concrete and repeatable on a hardware-in-the-loop (HIL) test bench.
- Expected results must define observable pass/fail criteria with explicit tolerances.
- Hazard severity levels must be categorized: `CRITICAL`, `HIGH`, `MEDIUM`, or `LOW`.

6.2 Self-Healing Error Handling Pipeline

1. **Regex Sanitization & Repair:** Strips markdown code fences (` ```json ... ``` `), extracts outermost JSON structures, and cleans trailing commas.
2. **Pydantic Schema Validation:** Validates fields against `GeneratedTestPlan` (ensuring 3–5 test cases, non-empty steps, valid severities).
3. **Corrective Feedback Retry:** On validation error, re-prompts the LLM with the exact validation error message and raw output (up to 3 retries).
4. **Deterministic Domain Fallback:** If network calls fail or no API key is provided, activates a domain generator that derives concrete test cases directly from the section specifications.

6.3 Duplicate Selection Submission Policy

Policy: *Content-Hash Idempotency with Explicit Force Override*.

Rationale: In regulated medical software QA, reproducibility is vital. Re-running non-deterministic LLMs on identical specifications creates duplicate, divergent test IDs, which risks audit confusion and wastes API tokens. By default, the system returns cached generations with `cached: true`, while offering `force_regenerate: true` when new test ideas are needed.

7. Live Staleness & Semantic Impact Detection

When retrieving test cases (`GET /api/test-cases/{id}`), the system compares the pinned content hashes recorded at generation time against the latest version's corresponding nodes.

- **FRESH:** All referenced nodes have identical hashes in the latest version.
- **STALE:** One or more referenced nodes have changed.
- **ORPHANED:** A referenced node was deleted in the latest manual version.

Semantic Impact Analysis

The system compares numeric values and thresholds across versions:

- `CRITICAL_THRESHOLD_CHANGE:` Alters numerical safety parameters (e.g. E3 emergency auto-deflate time reduced from 2.0s to 1.5s, or cuff inflation increment reduced from 40 mmHg to 30 mmHg).
- `SPECIFICATION_MODIFICATION:` Non-numeric wording or explanatory changes.

8. Decision Log (Required Questions)

1. What's the one part of this system most likely to silently give wrong results without erroring? How would you catch it?

Answer: The most dangerous failure mode is **parent-child misattribution caused by unstructured formatting anomalies** (e.g. an author inserting a bold sub-bullet that begins with numbers, or changing section number schemes to alphabetic prefixes). The parser would succeed without raising an exception, but would silently attach the section to the root or an incorrect ancestor.

How to catch it: Implement an automated **Hierarchy Invariant Verifier** that runs at the end of ingestion. It asserts mathematical tree invariants:

1. Section depth must equal numbering dot-depth ($Level(S) = Count(dots) + 1$).
2. Monotonicity checks: no section may have a child with a higher major prefix (a child of 3 cannot start with 4).
3. Visual indentation cross-check: horizontal coordinate $x_{indent}(child) \ge x_{indent}(parent)$.

2. Where did you choose simplicity over correctness because of time, and what would break first if this went to production as-is?

Answer: We chose simplicity by relying on **section number prefixes for logical node identity** (`sec_3_2` , `sec_4_2`) rather than training a continuous semantic embedding and graph-alignment model.

What breaks first in production: If an engineering editor inserts a new section 2.1 ("Regulatory Standards") and renumbers every subsequent section (so old 2.1 General Specs becomes 2.2), our matcher will compare old 2.1 against new 2.1, falsely reporting that "General Specs" was completely rewritten into "Regulatory Standards". A production system would combine section numbers with cosine similarity of headings to distinguish renumbered sections from modified sections.

3. Name one input (to your parser, your versioning matcher, or your LLM call) that you did not handle, and what your system does when it sees it.

Answer: We did not handle **scanned, raster-only bitmap PDFs containing zero digital text layers**. When our `pypdf` extractor encounters a purely scanned document, it extracts 0 text spans, produces an empty tree with 0 sections, and exits with a valid but empty `DocumentTree` .

Production remediation: The pipeline would incorporate an optical OCR fallback (such as Tesseract or AWS Textract) triggered automatically whenever extracted text character density falls below 50 characters per page.

9. What We Would Do Differently With More Time

1. **Interactive AST Diff Visualizer:** Render visual split diffs in a web UI highlighting exact numerical parameter edits in red/green.
2. **Hybrid Semantic + Graph Matcher:** Combine dense vector embeddings with hierarchy tree edit distance (Zhang-Shasha algorithm) to track sections across major reorganizations.

3. **Automated Impact-Scored Re-Generation:** When a requirement experiences a threshold change, auto-draft an updated test case diff (e.g. updating the expected result from 2.0s to 1.5s) and route it to QA leads for one-click approval.
